Arduino Microcontroller Concepts (CSE 4820)

Introduction to Arduino Microcontroller



Program: B. TECH., 7th Semester

SIKSHA 'O' ANUSHANDHAN (Deemed to be) UNIVERSITY

Chapter 6 — Course Map

How the topics connect

6.1 • Foundations

Sensor types
Digital / analog / pulse-width

6.2 • Movement

Tilt sensor
Digital switch

6.3 • Light

LDR
Voltage divider
Analog reading

6.4 • Motion

PIR
Digital output

6.5–6.6 • Distance

Ultrasonic + optical ToF

6.7–6.8 • Signals

Vibration + sound

6.9 • Temperature

TMP36
ADC → voltage → temperature

6.10 • DHT11

Temperature + humidity

6.1 Sensor Fundamentals

- A sensor detects a physical quantity or condition and produces an electrical signal that can be interpreted by a microcontroller.
- This Chapter deals with movement, light, motion, distance, vibration, sound and temperature.
- Sensor outputs may be digital, analog, pulse-width, or transmitted through a digital communication interface.
- Before programming, identify the sensor output and select the correct Arduino input method.
- Basic workflow: physical quantity → sensor → electrical signal → Arduino → processing → output/action.

6.1 Digital, Analog and Pulse-Width Signals

Choosing the correct Arduino input

Digital

HIGH / LOW
Tilt, PIR

Analog

Varying voltage
LDR, Piezo, TMP36, sound

Pulse Width

Pulse duration
Ultrasonic echo

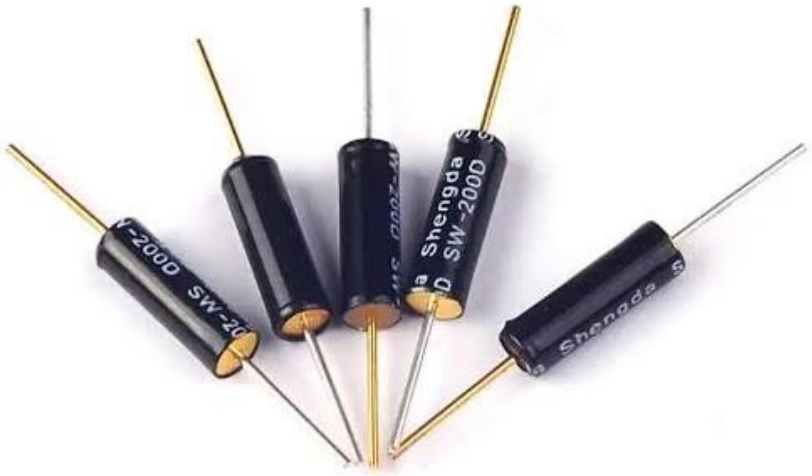
Digital Data

Sensor-specific communication
DHT11

6.2 Detecting Movement — Tilt Sensor

What is a tilt sensor?

- A tilt sensor is a switch-like sensor used to detect when an object is moved, tilted or shaken.
- Inside the sensor, a mechanical element changes the electrical contact when the sensor orientation changes.
- When the contact changes, the Arduino input changes state.
- It provides a digital-type indication rather than an exact angle measurement.
- A simple application is switching an LED or alarm when an object is tilted.





6.2 Tilt Sensor — Working Principle

Movement → switch state → Arduino decision

1 • NORMAL

Initial orientation
Switch in one state

2 • TILT / SHAKE

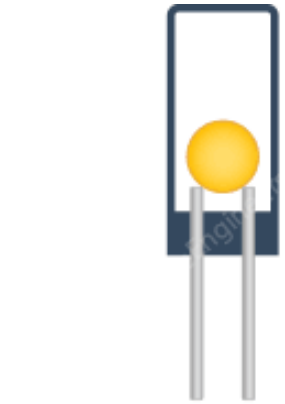
Mechanical element moves
Contact changes

3 • DIGITAL INPUT

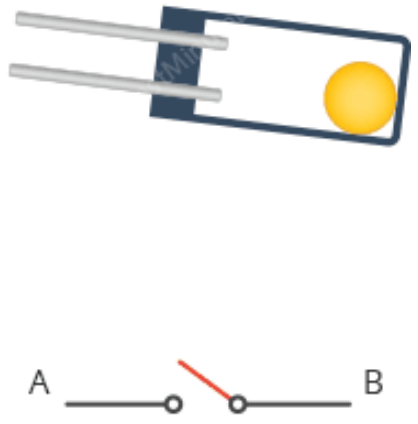
Arduino detects HIGH or LOW

4 • ACTION

LED, buzzer or control action



When the sensor is upright



When the sensor is tilted

6.2 Tilt Sensor — Interfacing

Using Arduino's internal pull-up

TILT SENSOR

One terminal → D2
Other → GND

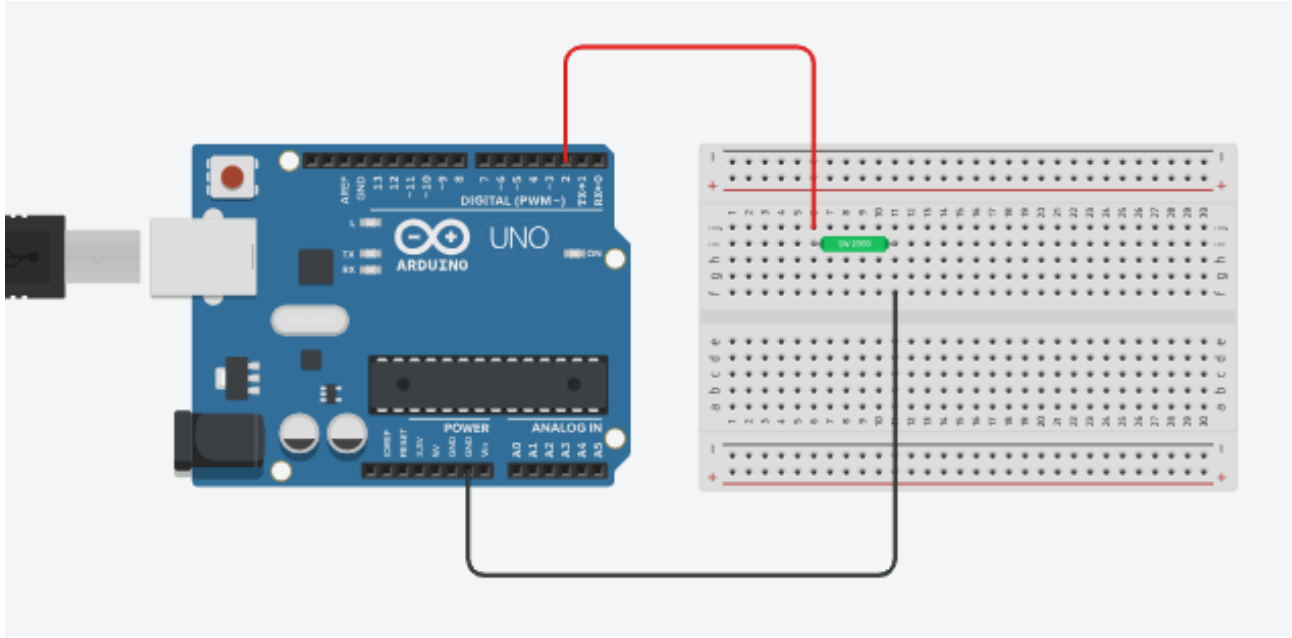
INPUT

pinMode(2, INPUT_PULLUP)

READ

digitalRead(D2)

- With INPUT_PULLUP, an open switch is normally HIGH.
- When the switch closes to ground, the input becomes LOW.
- This is active-low logic.



6.2 Tilt Sensor — Arduino Program

Complete example

```
const int tiltPin = 2;
const int ledPin = 13;

void setup() {
  pinMode(tiltPin, INPUT_PULLUP);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  int state = digitalRead(tiltPin);
  if (state == LOW)
    digitalWrite(ledPin, HIGH);
  else
    digitalWrite(ledPin, LOW);
}
```

- pinMode() configures the sensor input.
- INPUT_PULLUP provides an internal pull-up.
- digitalRead() obtains the sensor state.
- LOW indicates the switch is closed in this connection.
- The LED turns ON when the selected tilt condition is detected.

6.2 Tilt Sensor — Applications & Limitations

Where it can be used

APPLICATIONS

Anti-theft
Orientation indication
Movement detection

ADVANTAGES

Simple
Low cost
Easy interface

LIMITATIONS

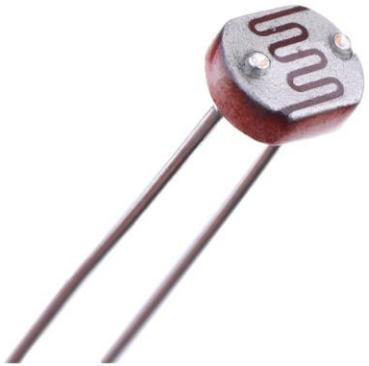
No exact angle
Mechanical bounce
Limited precision

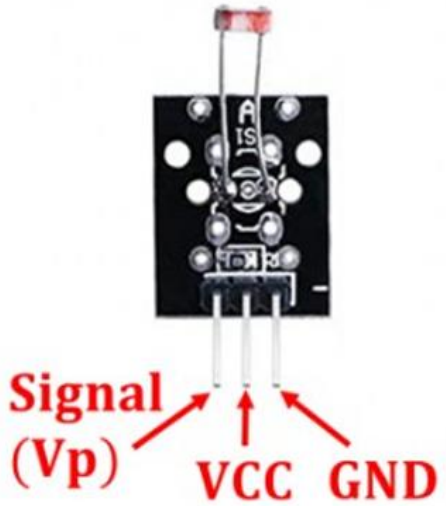
For accurate angle/orientation measurement, use an accelerometer or gyroscope.

6.3 Detecting Light — LDR / Photoresistor

What is an LDR?

- An LDR (Light Dependent Resistor), or photoresistor, is a resistive sensor whose resistance changes with light level.
- It can detect a change in illumination or estimate relative light level.
- Arduino analog inputs measure voltage, so the LDR is combined with a fixed resistor as a voltage divider.
- The divider converts resistance change into a measurable voltage.
- The voltage is read using analogRead().





Light → LDR resistance → divider voltage → ADC reading

6.3 LDR — Working Principle

LIGHT CHANGES

Illumination changes LDR resistance.

DIVIDER CHANGES

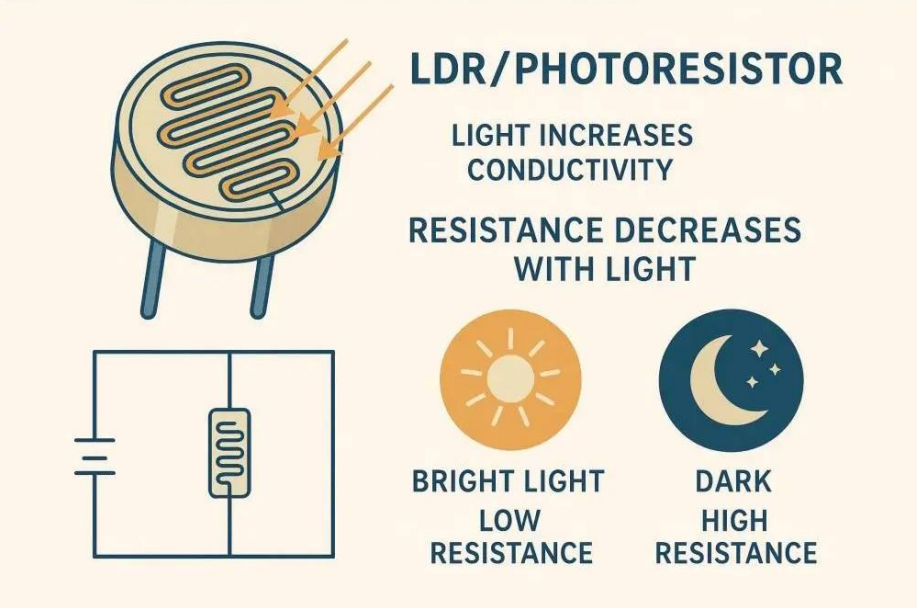
LDR + fixed resistor produce a different Vout.

ADC MEASURES

Arduino measures Vout.

PROGRAM USES IT

Display, threshold, control or scaling.



6.3 LDR — Voltage Divider

Circuit concept

VCC

Supply/reference

LDR

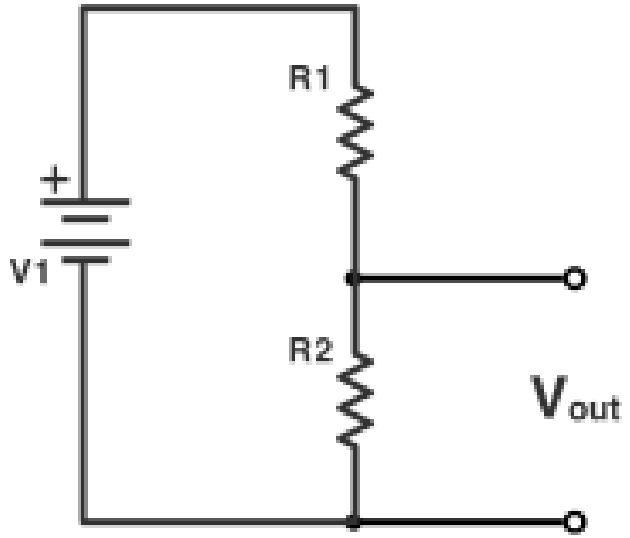
Variable resistor

VOUT

Divider junction
→ A0

FIXED R

Reference resistor
→ GND



$$V_{out} = V_1 \times R_2 / (R_1 + R_2)$$

6.3 LDR — Choosing a Fixed Resistor

Practical circuit design

- A useful starting point is a fixed resistor close to the LDR resistance in the darker part of its operating range.
- Measure the LDR with a multimeter under a representative lighting condition.
- A value around 10 kΩ can be a starting point for an LDR measuring around 10 kΩ under the chosen condition.
- The best value depends on the required light range and desired ADC sensitivity.
- After construction, measure actual dark and bright readings.



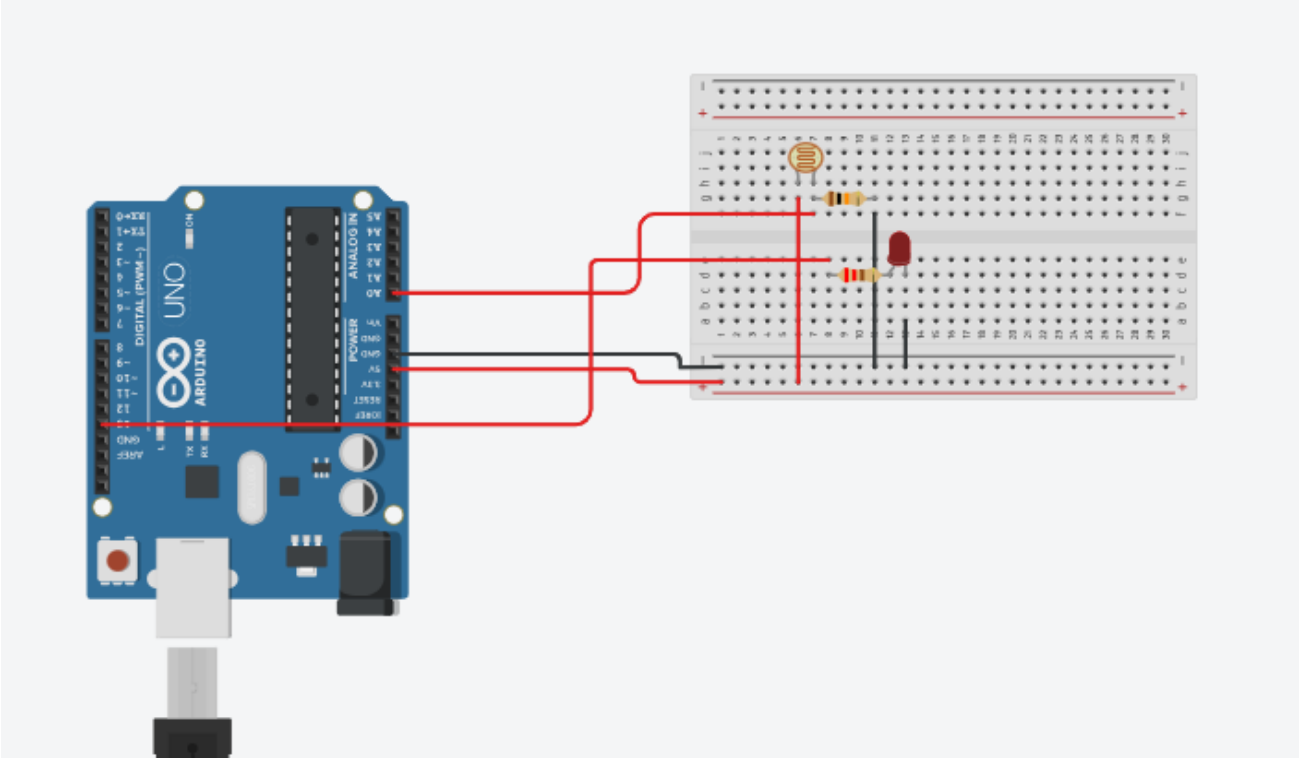
6.3 LDR — Arduino Program

Read the analog value

```
const int ldrPin = A0;
const int ledPin = 13;

void setup() {
 pinMode(ledPin,OUTPUT);
}

void loop() {
 int light = analogRead(ldrPin);
 if (light < 500)
 digitalWrite(ledPin,HIGH);
 else
 digitalWrite(ledPin,LOW);
 delay(100);
}
```

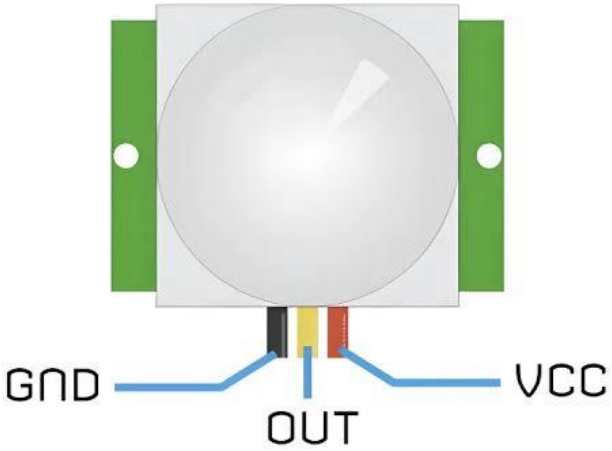


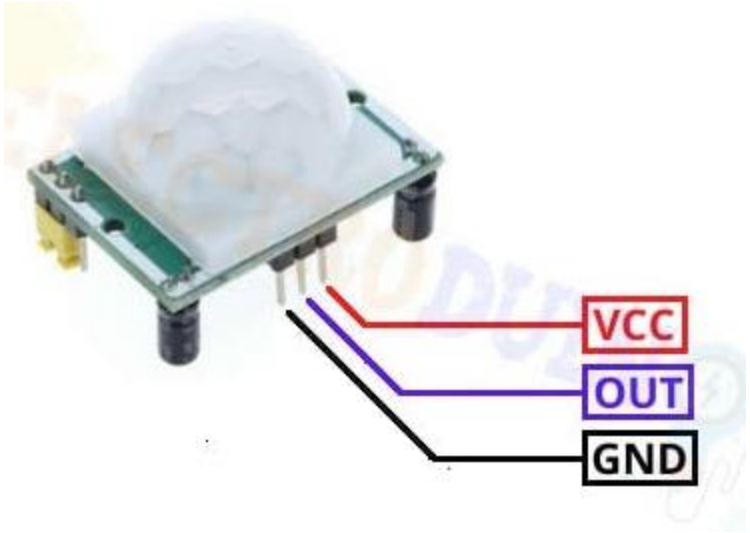
- analogRead(A0) returns the ADC reading.
- Change illumination and observe the reading.

6.4 Detecting Motion of Living Things — PIR

What is a PIR sensor?

- PIR means Passive Infrared.
- A PIR sensor detects changes in infrared radiation associated with moving people or animals.
- The sensor module provides a digital output representing detected motion.
- Arduino reads this output using digitalRead().
- Applications include room monitoring, automatic lighting and security systems.





6.4 PIR — Working Principle

How motion is detected

INFRARED ENERGY

Warm objects emit infrared radiation.

SENSOR ELEMENT

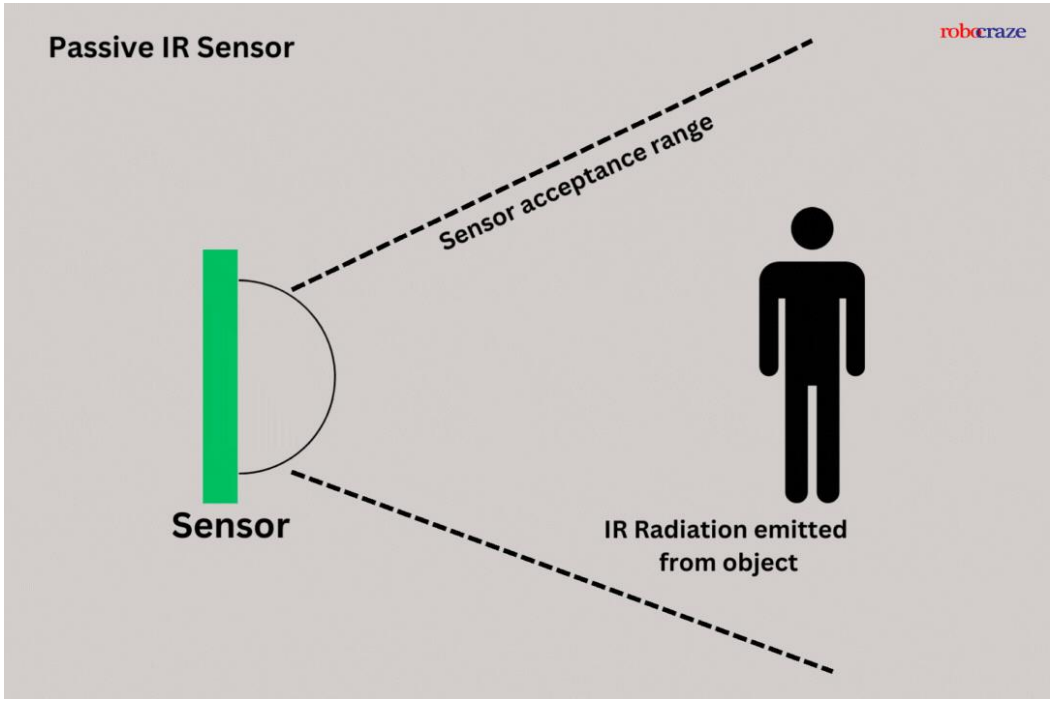
Responds to changes in infrared radiation.

PROCESSING

Module processes the sensor signal.

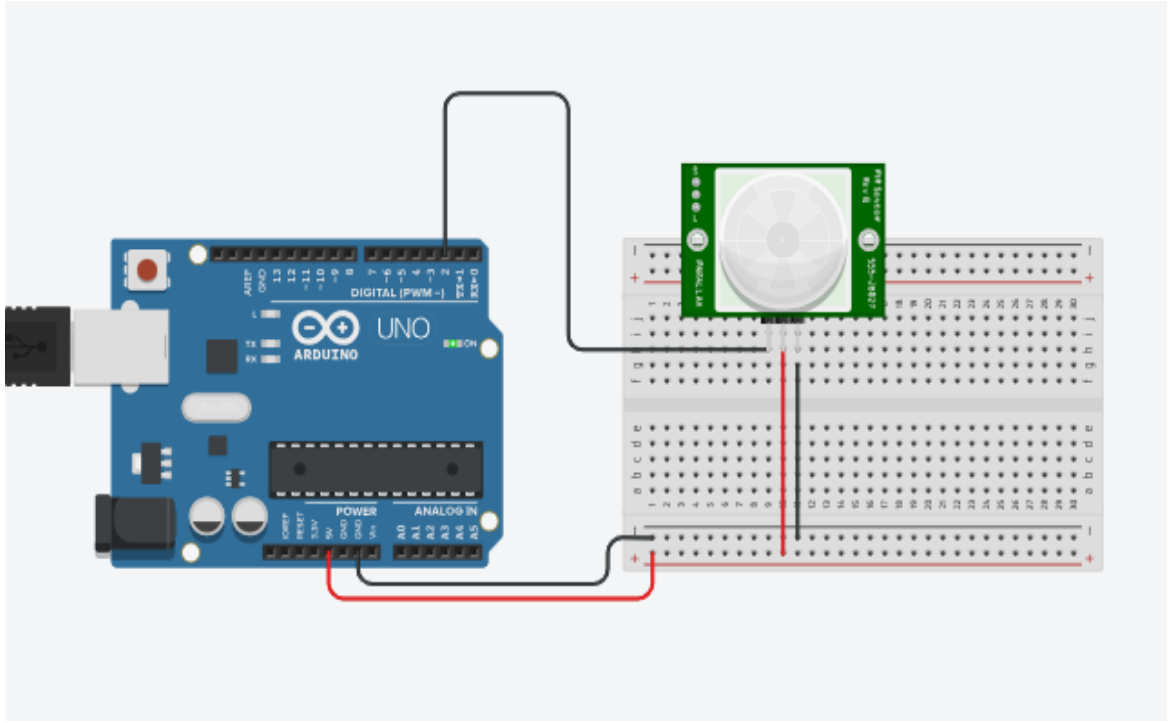
DIGITAL OUTPUT

Output changes when motion is detected.



6.4 PIR — Arduino Interfacing

Simple digital input




```
const int pirPin = 2;
const int ledPin = 13;
void setup() {
  pinMode(pirPin, INPUT);
  pinMode(ledPin, OUTPUT);
}
void loop() {
  int motion = digitalRead(pirPin);
  if (motion == HIGH) digitalWrite(ledPin, HIGH);
  else digitalWrite(ledPin, LOW);
}
```

- Connect PIR output to a digital input.
- A HIGH output can indicate motion, depending on the module.
- Verify the actual module's output behavior.
- The LED or another actuator can respond to the event.

6.5 Measuring Distance — Ultrasonic Sensor

What is ultrasonic distance measurement?

- An ultrasonic distance sensor measures distance using sound waves above the human audible range.
- The sensor sends an ultrasonic pulse toward an object.
- The pulse reflects from the object and returns to the sensor.
- Arduino measures the time between transmission and reception.
- Distance is calculated from the measured round-trip time.

HC-SR04 Ultrasonic Distance Sensor

VCC: Supplies power to the sensor.

GND: Provides the common ground connection with Arduino.

TRIG: Arduino sends a short trigger pulse to start the distance measurement. The sensor then generates an **ultrasonic burst** automatically.

ECHO: The sensor outputs a HIGH pulse whose duration represents the **round-trip travel time** of the ultrasonic wave.



6.5 Ultrasonic — Working Principle

Trigger → sound → reflection → echo

TRIGGER

Start a measurement.

TRANSMIT

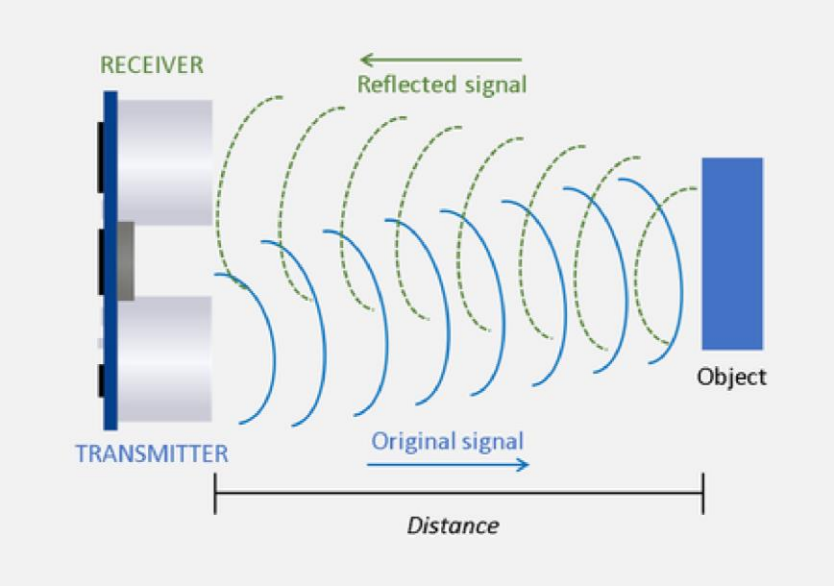
Pulse travels toward object.

REFLECT

Object reflects sound.

ECHO

Return time is measured.



6.5 Ultrasonic — Distance Formula

Why divide by two?

Distance = Speed of Sound × Time / 2

The sound travels to the object and then back.

```
distance_cm ≈ duration_µs / 29 / 2
```

The exact conversion depends on the sensor and assumed speed of sound.

6.5 Ultrasonic — pulseIn()

Measuring the echo pulse

```
duration = pulseIn(echoPin, HIGH);

// duration is in microseconds
// then convert time → distance
```

- pulseIn() measures the duration of a HIGH or LOW pulse.
- The returned duration is in microseconds.
- A timeout can be specified if an echo may not arrive.
- The measured duration is converted into distance.

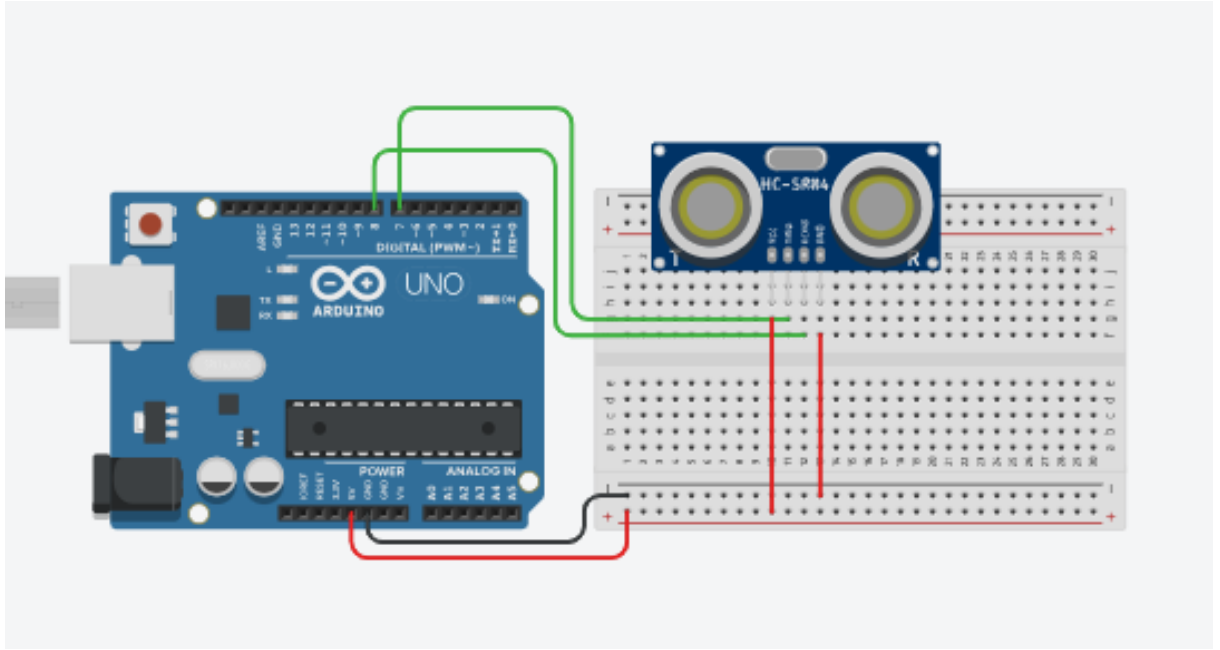
6.5 Ultrasonic — Arduino Program

Complete example

```
const int trigPin = 7;
const int echoPin = 8;

void setup() {
 pinMode(trigPin, OUTPUT);
 pinMode(echoPin, INPUT);
 Serial.begin(9600);
}

void loop() {
 digitalWrite(trigPin, LOW);
 delayMicroseconds(2);
 digitalWrite(trigPin, HIGH);
 delayMicroseconds(10);
 digitalWrite(trigPin, LOW);
 long duration = pulseIn(echoPin, HIGH);
 float distance = duration / 29.0 / 2.0;
 Serial.println(distance);
 delay(200);
}
```



- Trigger starts the measurement.
- Echo receives the returned pulse.
- pulseIn() measures echo duration.
- Duration is converted to approximate centimeters.
- The result can be displayed or used for control.

6.5 Ultrasonic — Worked Example

Understanding the calculation

Measured round-trip time: t = 2000 µs

Distance ≈ 2000 / 29 / 2

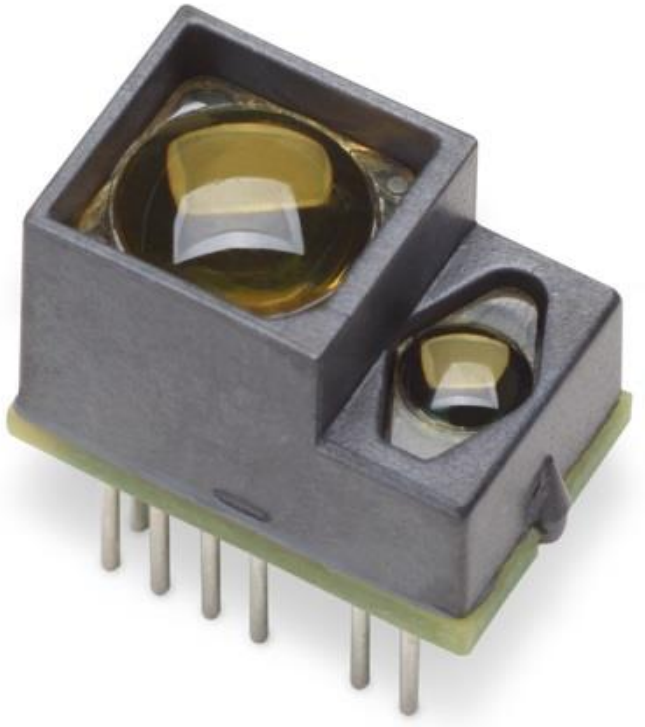
Distance ≈ 34.5 cm

- The measured time represents the complete out-and-back path.
- Dividing by two gives the one-way distance.
- Actual accuracy depends on sensor and environmental conditions.

6.6 Measuring Distance Precisely — Optical ToF

Time-of-flight using light

- An optical time-of-flight sensor measures distance using emitted light and the returning signal.
- The method uses light rather than sound.
- Optical ToF sensors can provide precise range information within their operating range.
- Field of view and target properties affect practical performance.
- A digital interface/library is commonly used to obtain the range value.



Ultrasonic ToF uses sound; optical ToF uses light.

6.6 Ultrasonic vs Optical ToF

Choosing a distance-sensing method

ULTRASONIC

Sound wave
Longer-range applications

OPTICAL ToF

Light signal
Precise range information

COMMON IDEA

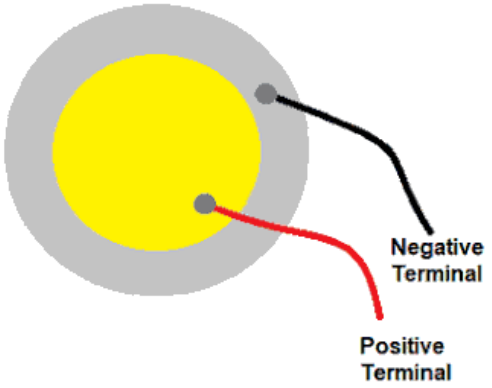
Both use time-of-flight
and convert it to distance.

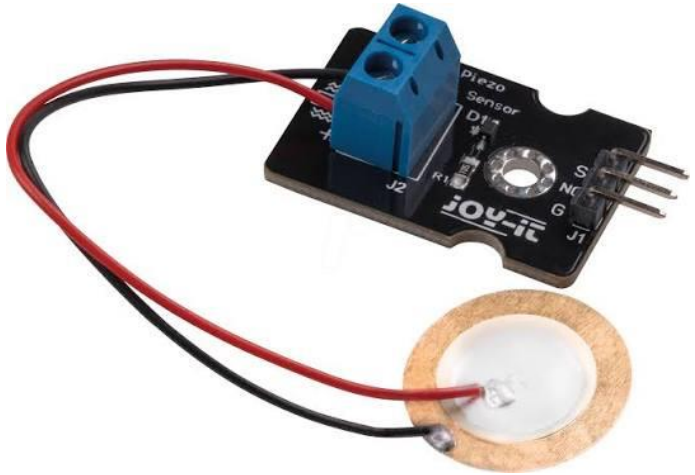
Choice depends on range, precision, target and environment.

6.7 Detecting Vibration — Piezo Sensor

What is a Piezo sensor?

- A Piezoelectric sensor produces an electrical signal when mechanically stressed or vibrated.
- It can be attached to a surface whose vibration needs to be detected.
- Arduino can sample the sensor voltage using an analog input.
- A threshold can decide whether vibration is strong enough to trigger an event.
- Applications include knock, vibration and impact detection.





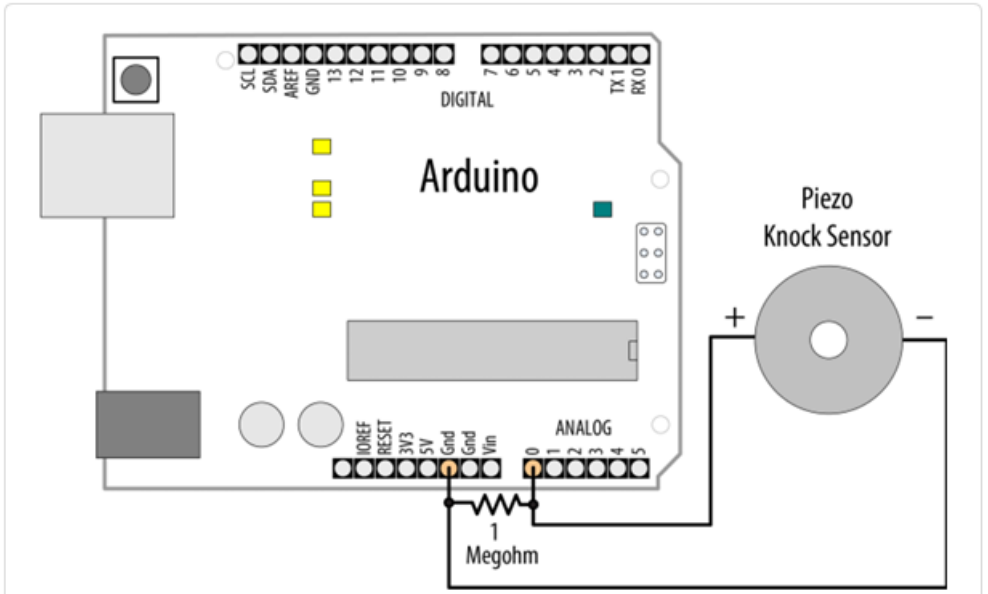
Vibration → mechanical stress → electrical signal → analog reading.

6.7 Piezo — Threshold Detection

Turning an analog signal into an event

```
const int sensorPin = A0;
const int threshold = 500;
void setup()
{
 pinMode(LED_BUILTIN, OUTPUT);
 Serial.begin(9600);
}
void loop()
{
 int value = analogRead(sensorPin);
 Serial.println(value);
 if (value >= threshold)
 {
  digitalWrite(LED_BUILTIN, HIGH);
 }
 else
 {
  digitalWrite(LED_BUILTIN, LOW);
 }

 delay(100);
}
```



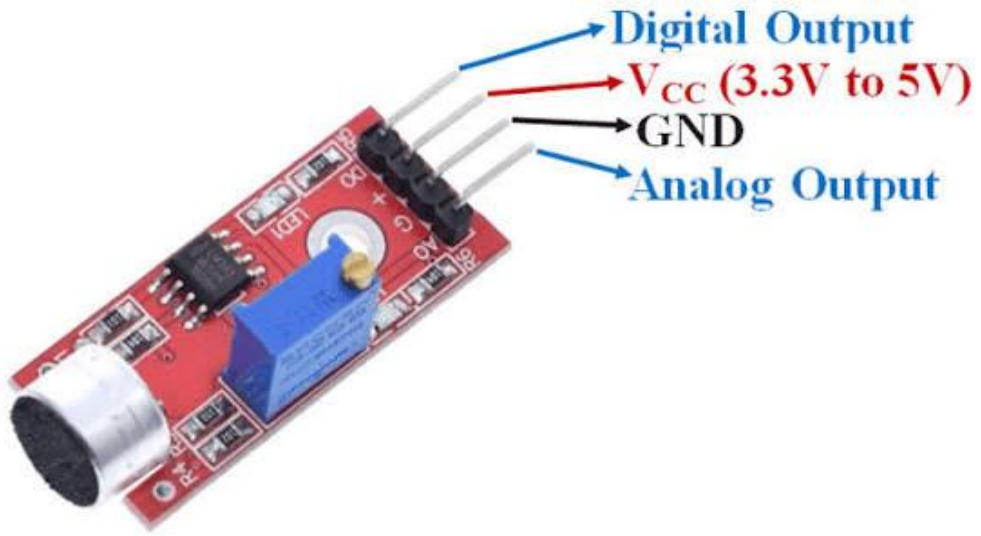
Knock sensor connections

- analogRead() obtains the sensor reading.
- Threshold represents the minimum value treated as a vibration event.
- Select the threshold from observed readings.
- Noise can cause false triggers; filtering or confirmation may help.

6.8 Detecting Sound — Microphone

From sound pressure to an Arduino signal

- A microphone converts sound-induced pressure variations into an electrical signal.
- The microphone output is an analog waveform.
- The waveform can contain a DC offset, so its center is not necessarily zero.
- Arduino samples the signal using an analog input.
- Amplitude, averaging and RMS-type calculations can estimate signal strength.



A raw microphone ADC value is not automatically a calibrated sound-pressure level.

6.8 Sound — DC Offset and Amplitude

Understanding the waveform

DC OFFSET

Signal midpoint is around a non-zero ADC value.

WAVEFORM

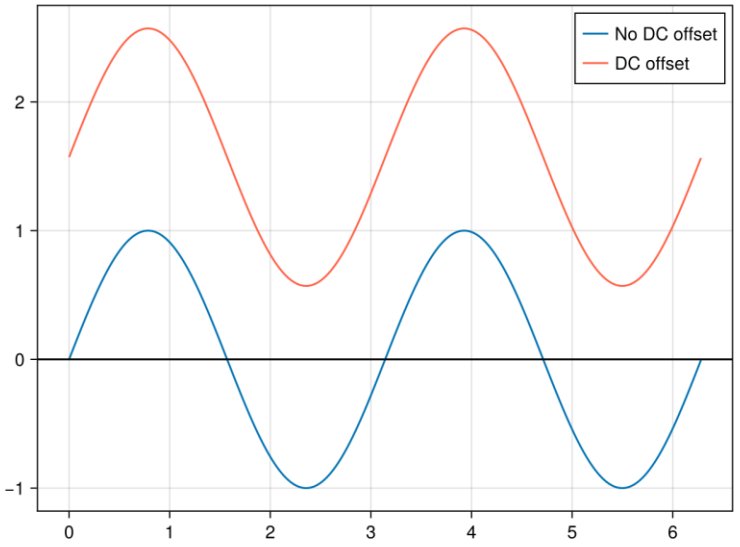
Sound moves the signal around the midpoint.

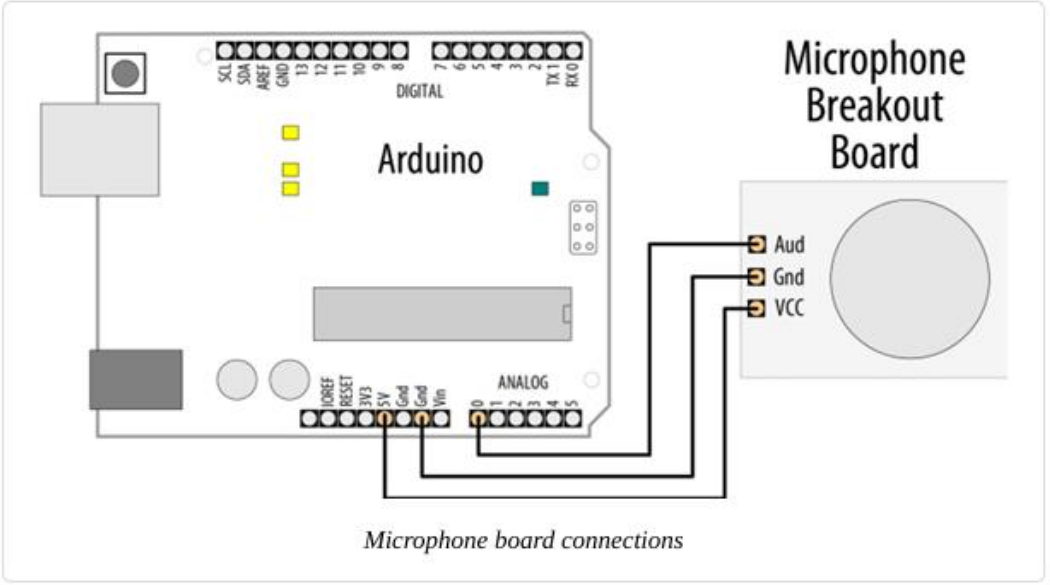
AMPLITUDE

Variation size indicates signal strength.

PROCESSING

Peak-to-peak, averaging or RMS can summarize it.





Microphone board connections

6.8 Sound — Averaging and RMS

Estimating signal strength

RMS concept: square → average → square root

- A single sample changes rapidly because sound is a varying waveform.
- A group of samples provides more useful information.
- Peak-to-peak measures the range between high and low values.
- RMS provides a mathematical measure of effective signal magnitude.
- Accurate conversion to physical sound level requires calibration.

Calibration connects an electrical measurement to a known sound-level reference.

6.8 Sound — Practice Question

Sound detection using a microphone

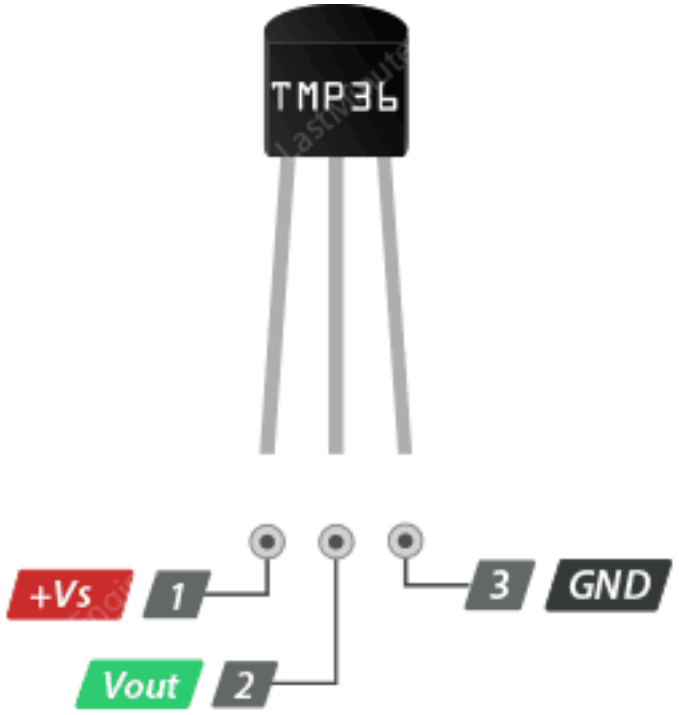
- Objective: Observe and process an analog microphone signal.
- Connect the microphone module to an analog input.
- Display raw readings using Serial Monitor/Plotter.
- Observe the signal in a quiet environment.
- Create sound near the microphone and observe amplitude change.
- Experiment with averaging or thresholding to create a sound-event detector.

Do not report a calibrated dB value unless the system has been calibrated.

6.9 Measuring Temperature — TMP36

Analog temperature sensing

- TMP36 is an analog temperature sensor that produces a voltage related to temperature.
- Arduino reads the sensor voltage through an analog input.
- The ADC reading is first converted into voltage.
- The sensor's voltage-temperature relationship is then used to calculate temperature.
- The calculated temperature can be displayed or compared with a threshold to control a device.



Measurement chain: ADC count → voltage → temperature.

6.9 TMP36 — Measurement Chain

Understanding the conversion

1 • ADC COUNT

analogRead() gives a numerical value.

2 • VOLTAGE

Convert ADC count using actual reference voltage.

3 • SENSOR RELATION

Apply TMP36 voltage-temperature relationship.

4 • RESULT

Display °C/°F or use for control.

Use the actual reference voltage and the correct sensor relationship.

6.9 TMP36 — Arduino Program

Basic temperature measurement

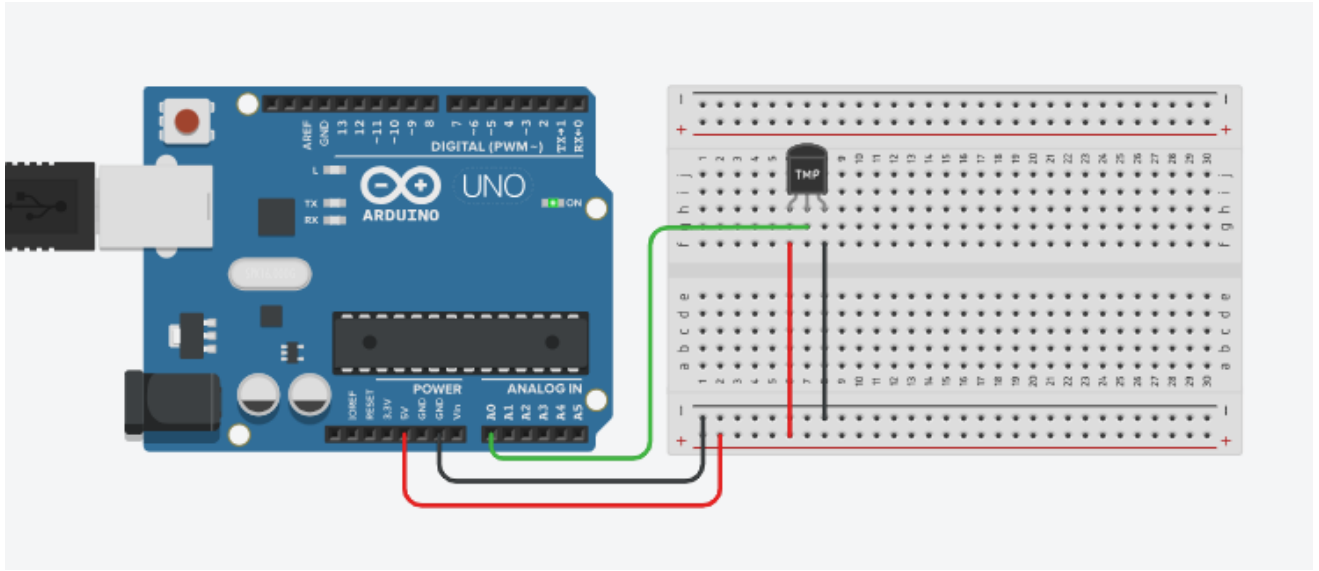
```
const int inPin = A0;
void setup()
{
  Serial.begin(9600);
}
void loop()
{
  // Read the analog value
  int value = analogRead(inPin);

  // Convert ADC reading to millivolts
  float millivolts = (value / 1024.0) * 5000;

  // TMP36 voltage-temperature relationship
  float celsius = (millivolts - 500) / 10;

  // Display temperature in Celsius
  Serial.print("Temperature: ");
  Serial.print(celsius);
  Serial.println(" C");

  delay(1000);
}
```



- Read the analog output.
- Convert the ADC count using the actual reference voltage.
- Apply the sensor-specific temperature equation.
- Display the result with the correct unit.
- Use the calculated temperature for monitoring or control.

6.9 Temperature — Threshold Control

Using temperature for control

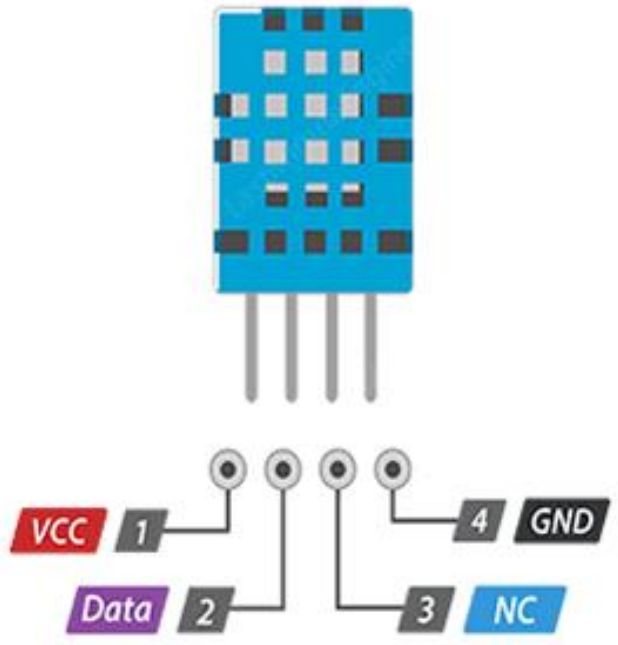
```
if (temperature >= 30.0) {
  digitalWrite(LED_PIN, HIGH);
} else {
  digitalWrite(LED_PIN, LOW);
}
```

- A measured temperature can be compared with a threshold.
- The threshold can control an LED, fan, alarm or other actuator.

6.10 Temperature & Humidity — DHT11

Temperature and humidity sensing

- DHT11 is a digital sensor that provides temperature and relative-humidity measurements.
- It communicates data digitally rather than providing a simple analog voltage like TMP36.
- Arduino reads the data using the appropriate DHT11 interface/library.
- Measured temperature and humidity can be displayed on the Serial Monitor or another display.
- Verify the module pinout, supply requirements and data connection before programming.



TMP36 → analog measurement; DHT11 → digital sensor data.

6.10 DHT11 — Interfacing

Basic connection

VCC

Connect to specified supply.

DATA

Connect to an Arduino digital pin.

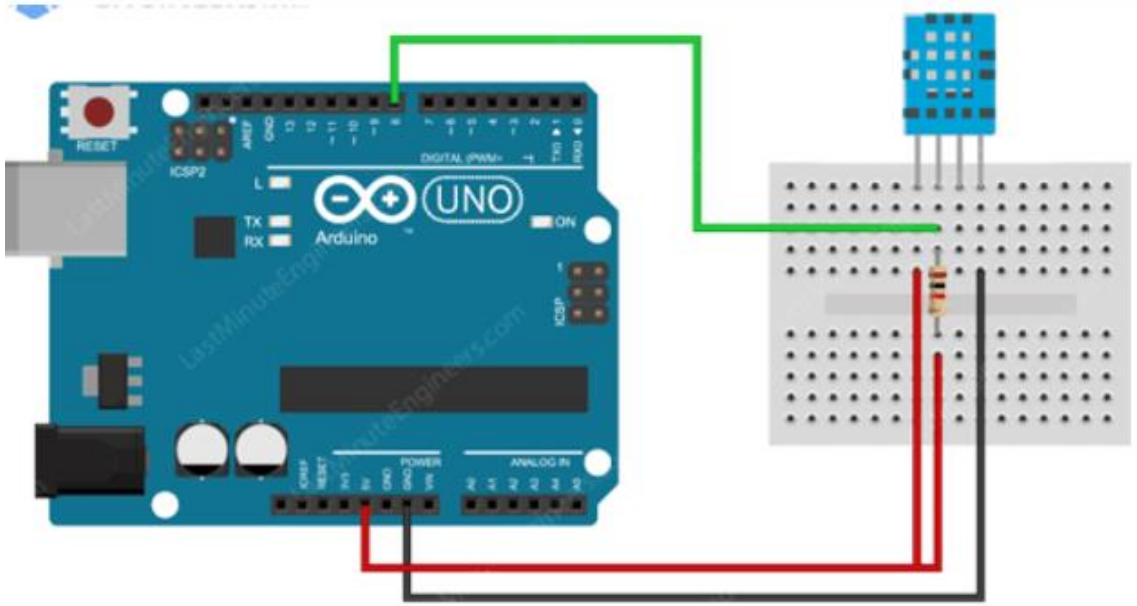
GND

Connect to common ground.

ARDUINO

Initialize → read → check → display

- Check the module's exact pin order before wiring.
- Use the appropriate sensor library/interface.
- Read both temperature and relative humidity.
- Check whether the reading is valid before displaying it.



Calibration

Making sensor measurements trustworthy

- Calibration establishes the relationship between a sensor output and a known reference.
- Offset error makes readings consistently too high or too low.
- Scale error changes the slope between sensor output and the reference quantity.
- Calibration is important when an electrical signal must be converted into a physical unit.
- Accurate sound-level measurement requires calibration against a known reference.
- Calibration should be performed under controlled and repeatable conditions.

Calibration connects sensor output to a trustworthy physical measurement.

Interpreting Sensor Readings

A raw number needs context

- Identify the signal type: digital state, ADC count, voltage, pulse duration or digital sensor value.
- Identify the physical quantity being measured.
- Apply the correct conversion equation or sensor relationship.
- Use the correct units.
- Consider range, resolution, accuracy and environmental effects.
- Check readings for noise, impossible values and unexpected changes.
- Use repeated measurements or averaging when appropriate.

Never assume that an ADC value directly equals the physical quantity.

How to Choose the Arduino Function

A quick decision guide

ON / OFF?

digitalRead()
Tilt, PIR

VOLTAGE VARIES?

analogRead()
LDR, Piezo, TMP36

PULSE DURATION?

pulseIn()
Ultrasonic echo

DIGITAL DATA?

Sensor-specific interface
DHT11

Top 10 Questions — Chapter 6

Revision / Viva / Tutorial

1. What is an LDR and how does it work?
Taxonomy: Remember (L1)

2. Explain how an LDR is interfaced with Arduino using a voltage divider.
Taxonomy: Understand (L2)

3. Differentiate between digital and analog sensors with examples.
Taxonomy: Understand (L2)

4. Explain how a tilt sensor can be interfaced with Arduino. Taxonomy:
Apply (L3)

5. Explain how a Piezo sensor can be used for vibration detection. Taxonomy:
Apply (L3)

6. Explain how an ultrasonic sensor measures distance using TRIG and ECHO signals. Taxonomy:
Understand (L2)

Top 10 Questions — Chapter 6

Revision / Viva / Tutorial

7. An ultrasonic sensor measures an echo duration of 2000 µs. Calculate the approximate distance.
Taxonomy: Apply (L3)

8. Explain the voltage-temperature relationship of the TMP36 sensor.
Taxonomy: Understand (L2)

9. Why is calibration important when interpreting sensor readings?
Taxonomy: Analyze (L4)

10. Design an Arduino-based system that activates an LED when a sensor value exceeds a selected threshold.
Taxonomy: Create (L6)